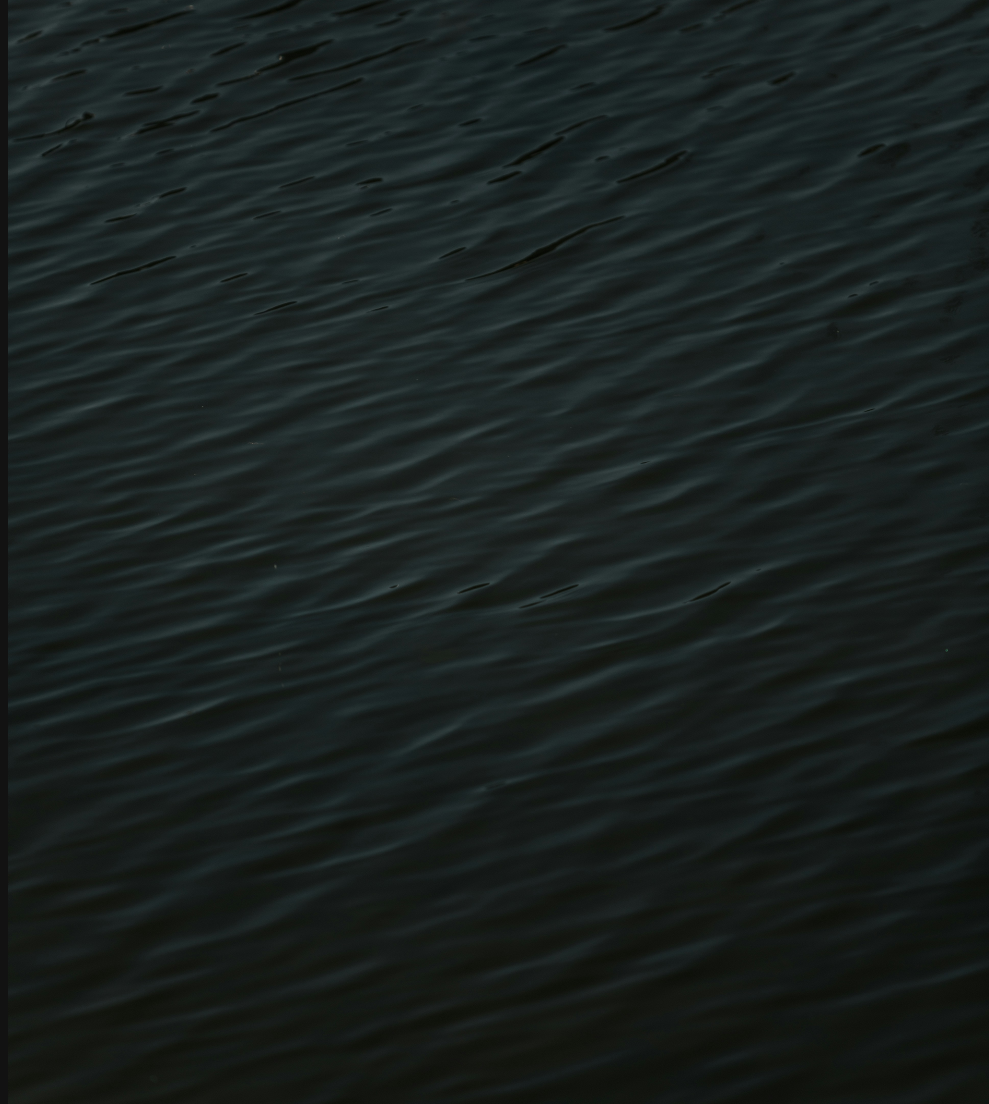


Cifras de Fluxo

Gerência e Segurança de Redes



Objetivo de Aprendizagem

- Conhecer o conceito de cifra de fluxo

Agenda

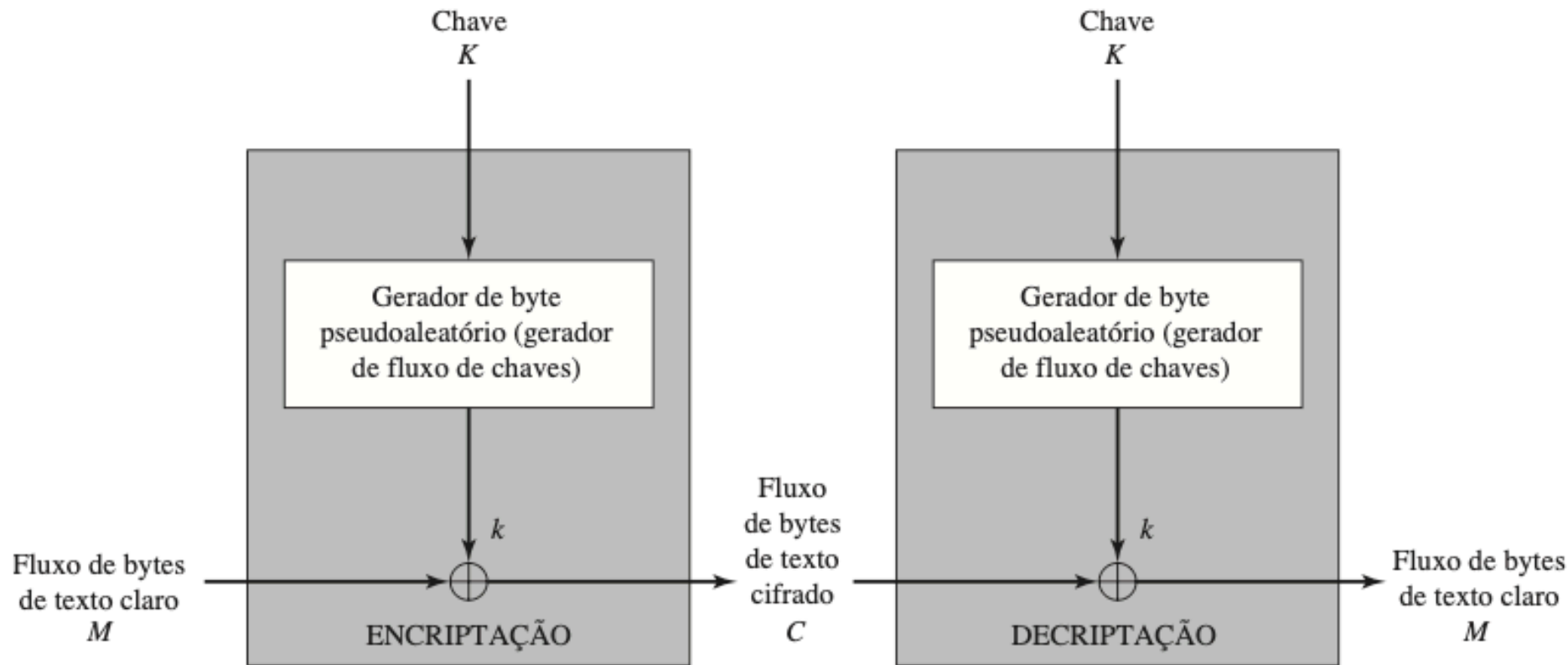
- Estrutura
- Cifras típicas
- Aplicações
- Vulnerabilidades

Estrutura

Estrutura

Cifras de fluxo

- Criptografa um *byte* por vez
- Comumente aplicado a canais de comunicação ou aplicações cliente/servidor
- Estrutura básica:
 - Chave de entrada (K)
 - Fluxo de *bits*, ou fluxo de chave (k)
 - XOR
 - Texto claro
 - Texto cifrado



Exemplo

	1	1	0	0	1	1	0	0	Texto claro
	0	1	1	0	1	1	0	0	Fluxo de chave
XOR	1	0	1	0	0	0	0	0	Texto cifrado
	1	0	1	0	0	0	0	0	Texto cifrado
	0	1	1	0	1	1	0	0	Fluxo de chave
XOR	1	1	0	0	1	1	0	0	Texto claro

Fluxo de chave

Key stream

Um fluxo **pseudo-aleatório** é imprevisível sem conhecimento da chave de entrada

Fluxo de chave

Key stream

Um fluxo genuinamente **aleatório** é aquele que é totalmente imprevisível.
One Time Pad.

**Por que utilizar
um fluxo pseudo-
aleatório?**

Cifras de Fluxo

Características

- O gerador de números pseudo-aleatório utiliza funções determinísticas e periódicas para geração do fluxo
- Quanto maior o período, maior a dificuldade de criptoanálise
- Quanto mais próximo do fluxo genuinamente aleatório, maior a dificuldade de criptoanálise
- Chave K de pelo menos 128 bits
- Segurança das Cifras de Fluxo é comparável às Cifras de Bloco

Cifras de Fluxo

Vantagens

- Implementação simplificada
- Baixa demanda computacional
 - DES, 9Mbps
 - 3DES, 3Mbps
 - RC4, 45Mbps
- Reutilização de chaves

Cifras de Fluxo

Aplicações

- Canais de comunicação
- Cliente/Servidor
- Criptografia de unidades de armazenamento

Cifras Típicas

Cifras de Fluxo

Exemplos

- Cifra de *Vernam*
- Salsa20
- RC4

Cifra de *Vernam*

- Uma das primeiras CF (*Gilbert Vernam*, 1917)
- Chave e texto claro do mesmo tamanho
- XOR bit a bit entre texto claro e chave
- Generalização do *One Time Pad*

Salsa20

- CF proposta por *Daniel Bernstein* em 2005
- Segurança e eficiência
- Chaves de 128, 192 e 256 *bits*
- Fundamenta-se no uso de *nonces*

Salsa20

Number used Once

Número aleatório utilizado uma única vez em um algoritmo criptográfico

nonce

Propriedades

- Unicidade
- Utilização única
- Imprevisibilidade

nonce

Aplicações

- Prevenção a ataques de repetição
 - Evita que mensagens de autenticação antigas sejam reutilizadas por atacantes
- *Proof-of-Work*
 - `nonce` são aplicados pelos mineradores para encontrar um *hash* que atenda a uma dificuldade específica
- Cifras de fluxo

UUID

Universally Unique Identifier

- RFC 4122
- Identificador de 128 bits
- Baixa probabilidade de colisão (repetição)

UUID

Versões

- **UUIDv1** Baseado no *timestamp* e no endereço MAC da máquina (previsível)
- **UUIDv4** Gerado usando números aleatórios ou pseudo-aleatórios. É a versão mais comum (imprevisível)
- Outras versões (v3, v5) são baseadas em *hash* de namespaces e nomes

nonce e UUID

Característica	Nonce (Requisito)	UUID (Realidade)	É um Bom Nonce?
Unicidade	Obrigatória	Alta probabilidade (depende da versão). UUIDs são projetados para serem únicos, mas colisões teoricamente são possíveis (embora muito raras).	✓ Geralmente Sim
Uso Único	Obrigatório	O UUID em si não impõe isso. Cabe ao sistema/implementação garantir que um UUID específico não seja reutilizado como nonce.	✓ Sim (se bem gerido)
Imprevisibilidade	Obrigatória	Depende da Versão. UUIDv1 é previsível . UUIDv4 é imprevisível (se usar CSPRNG).	UUIDv4: ✓ Sim UUIDv1: ✗ Não

Salsa20

- Utiliza um `nonce` de 64 bits como entrada junto com uma chave secreta

$$ChaveSecreta + nonce + Mix = FluxoDeChaves$$

$$FluxoDeChaves \oplus TextoClaro = TextoCifrado$$

Salsa20

Propriedades

- Imprevisibilidade
- Pseudoaleatoriedade

RC4

Rivest Cipher 4

- Criado por *Ron Rivest* para RSA em 1994
- Tamanho de chave variável
- Fundamenta-se na permutação aleatória
- Baixa demanda computacional
 - 8 a 16 operações de processador por *byte* de saída

RC4

Aplicações

- TLS 1.0
- WEP (*Wired Equivalent Privacy*)
- WPA (*WiFi Protected Access*)

RC4

Funcionamento

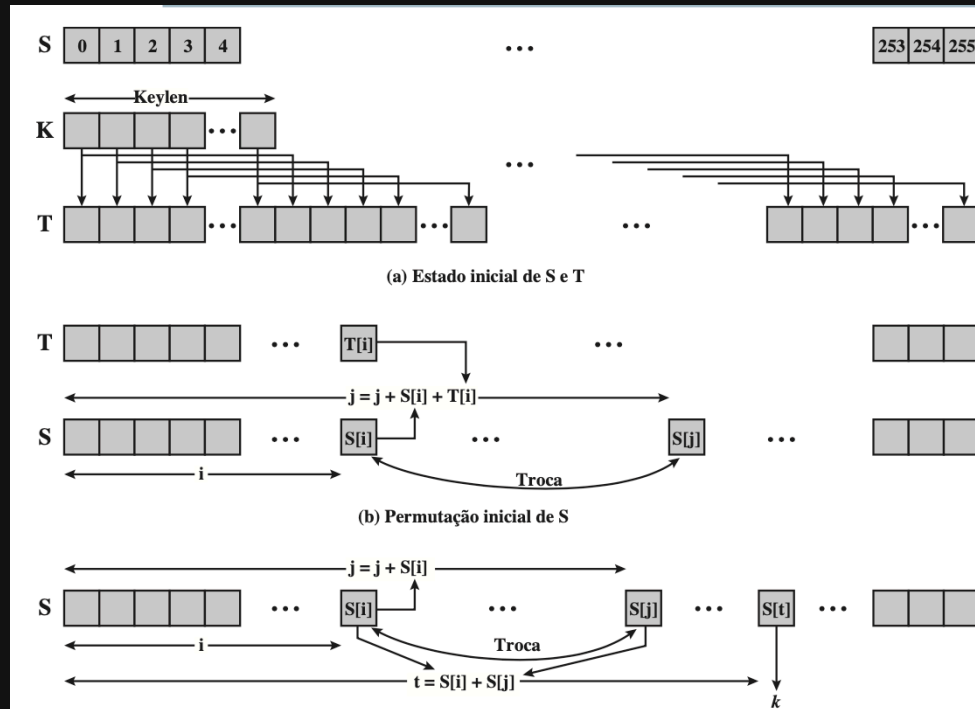
- Chave K possui tamanho variável entre 1 a 256 bytes (8 a 2048 bits)
- A chave inicializa o vetor de estados S de 256 bytes

$$S[0] = 0,$$

$$S[1] = 1,$$

...

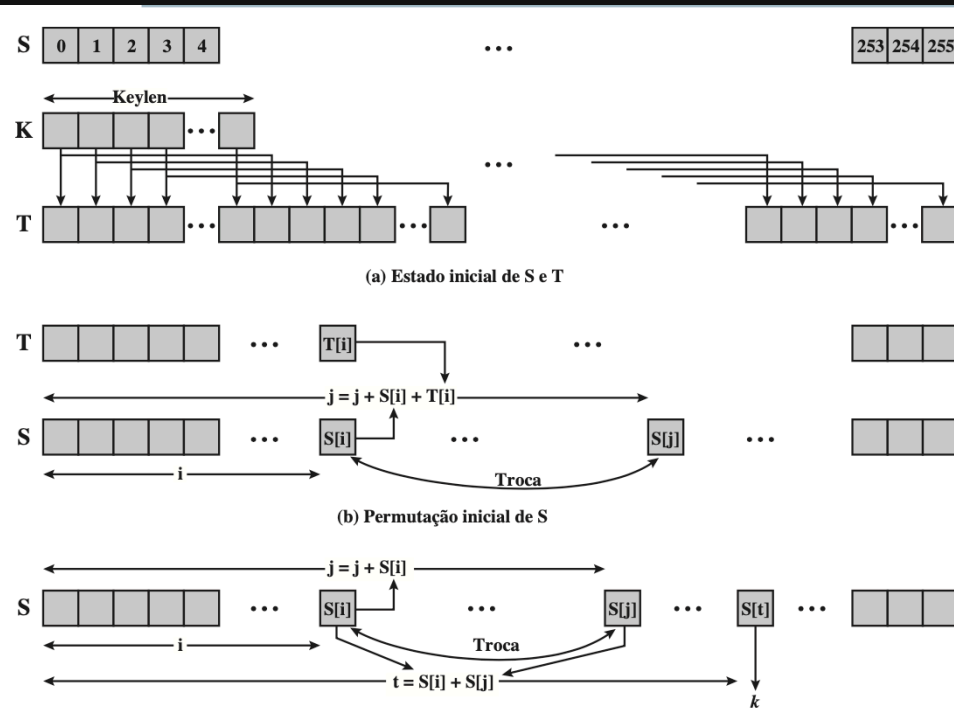
$$S[255] = 255$$



RC4

T

- Se K tem 256 bytes, um vetor temporário, T , é inicializado com o valor de S
- Caso contrário, K é replicado em T até que tenha o mesmo tamanho de S

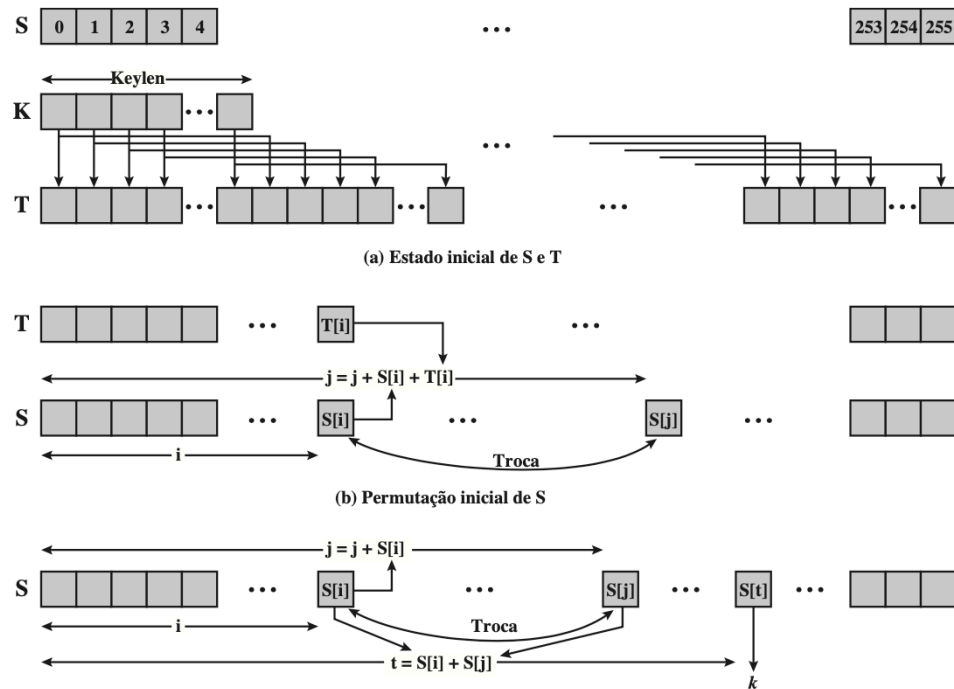


RC4

T

■ Resumo em pseudo-código:

```
1  for i = 0 to 255
2      S[i] = i;
3      T[i] = K[i mod keylen];
```

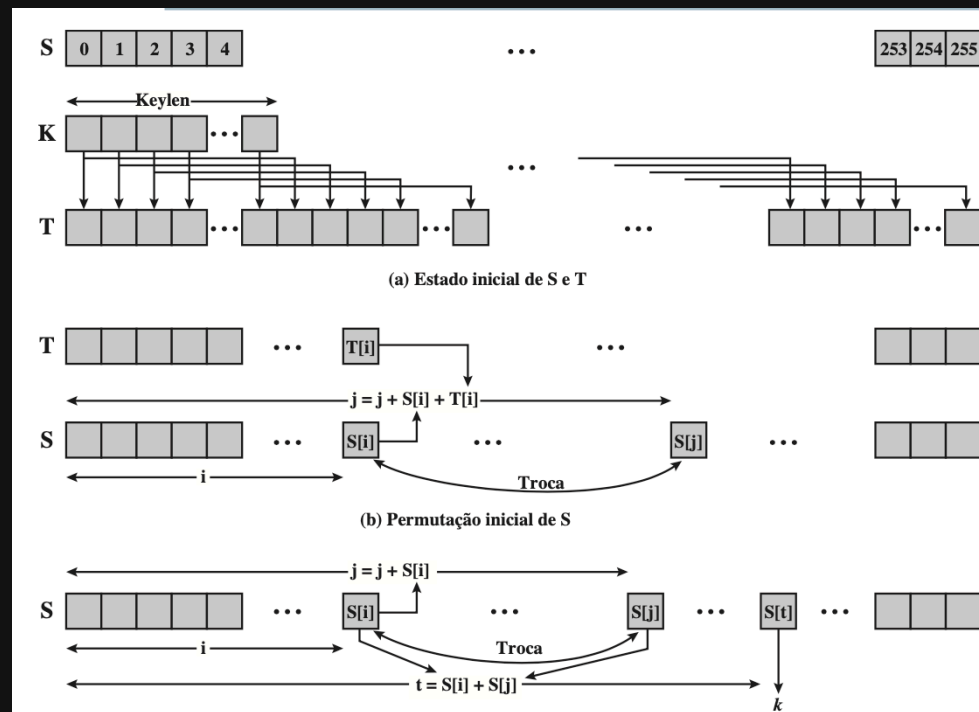


RC4

Permutações

- A função de T é causar permutações em S , conforme abaixo:

```
1  j = 0;  
2  for i = 0 to 255 do  
3      j = (j + S[i] + T[i]) mod 256  
4      Swap (S[i], S[j])
```

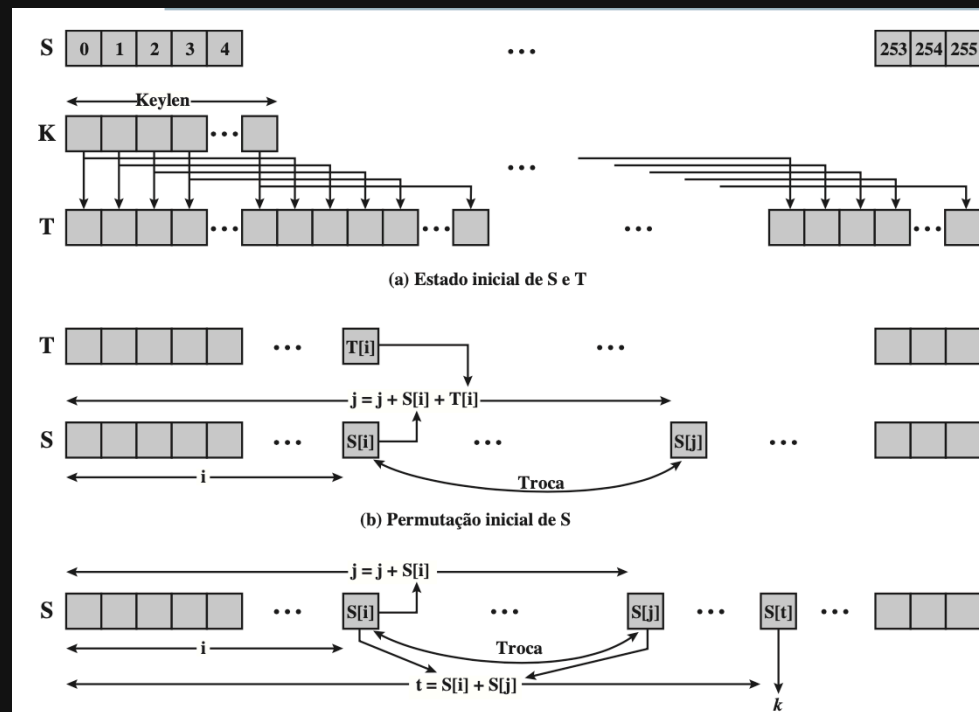


RC4

Fluxo de chaves

- O fluxo é gerado percorrendo cada elemento de S e realizando as permutações

```
1  i, j = 0;  
2  while True  
3      i = (i + 1) mod 256;  
4      j = (j + S[i]) mod 256;  
5  Swap (S[i], S[j])  
6  t = (S[i] + S[j]) mod 256;  
7  k = S[t]
```



RC4

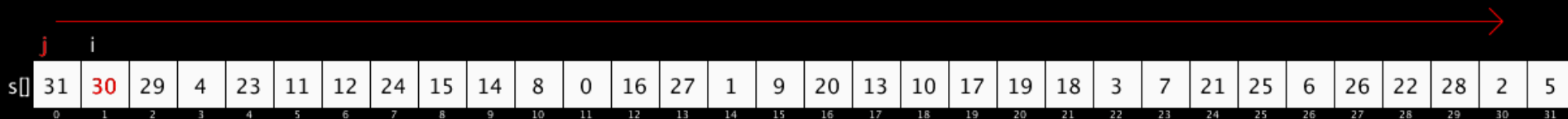
Animação com State de 32 bytes

Add one to i

Add s[i] to j

Swap s[i] and s[j]

Add s[i] and s[j] and output value in s[s[i] + s[j]]



RC4

Fragilidades

O RC4 tem inicialização frágil. Os primeiros *bytes* do fluxo de chaves tem correlação com a chave de inicialização

RC4

WEP

A fragilidade do RC4 no WEP é um dos casos mais famosos de falhas de segurança na história. O WEP não apenas usava o RC4, mas o fazia de forma incorreta.

WEP

Fragilidades

- Vetor de inicialização (IV) de apenas 24 bits e reutilizado frequentemente
- Chave composta de IV + chave secreta
- A partir dos dois primeiros *bytes* do fluxo de chave a chave poderia ser encontrada a partir da coleta de dados na rede sem fio

WEP

Fragilidades

Pesquisadores da AT&T e Rice *University*, juntamente com os desenvolvedores do `AirSnort` implementaram o algoritmo que explora essa vulnerabilidade e consegue derivar a chave após a coleta de cerca de 16 milhões de pacotes. Assim, surgiu o `AirSnort`. Baseada no mesmo algoritmo, tempos depois, foi desenvolvida outra ferramenta com o mesmo objetivo: o `WepCrack`

RC4

Resumo

- Formalmente, nunca houve a quebra da cifra RC4
- Porém, as falhas de implementação no WEP difundiram uma imagem negativa da cifra de fluxo
- Atualmente, o RC4 é considerado **obsoleto**
- Removido da TLS desde a versão 1.0

Referências

- NIST Terms `nonce`
- UUID Generator
- RC4 no WEP

Referências

- Capítulo 7. Criptografia e Segurança de Redes. William Stallings. 6ª. Edição. Editora Pearson.
 - Seções 7.4 e 7.5.
- Capítulo 6. Criptografia e Segurança de Redes. William Stallings. 4ª. Edição. Editora Pearson.
 - Seção 6.3.

José Roberto Bezerra

 jbroberto@ifce.edu.br

 [jbroberto76](#)

Powered by  Slidev

Foto da capa by [Mona Bernhardsen](#) na [Unsplash](#)